REPORTE FORENSE — Taller Docker

Contenerización, Seguridad de Runtime y Estrés de cgroups

Campo	Valor
Proyecto	`eco-air-backend`
Imagen	`eco-air-backend:1.0.0`
Estudiante	Hector Alejandro Riascos Insuasty
ID del estudiante (STUDENT_ID)	`871752`
Fecha de la auditoría	2026-10-20
Entorno	Docker Engine 29.7.2 (Linux / WSL2)

---

1. Justificación del taller

El backend `eco-air-backend` procesa **alertas ambientales críticas** que llegan en ráfagas continuas desde los sensores de una estación meteorológica. Por su naturaleza, el servicio corre en un servidor compartido donde otros procesos del sistema operativo también necesitan recursos. Dos riesgos hacen obligatorio un encapsulamiento estricto:

1. **Riesgo de privilege escalation.** Si el contenedor se ejecuta con privilegios de `root`, un atacante que comprometa la JVM obtiene `root` dentro del contenedor y, en configuraciones débiles, una vía hacia el host. La mitigación es ejecutar el proceso `java` con un usuario sin privilegios (`sensor_user`, UID 10001), de modo que la superficie de ataque se reduzca a un usuario de servicio.
2. **Riesgo de agotamiento de memoria.** La JVM, por defecto, tiende a acaparar toda la RAM disponible para su heap. En un servidor de estación meteorológica con recursos limitados, esto mataría otros procesos críticos. La mitigación es imponer una cuota estricta de memoria mediante **cgroups** (`-m`), de forma que, si la JVM excede el límite, el *OOM killer* del kernel termine el proceso (SIGKILL, exit 137) en lugar de colgar el servidor completo.

Por ello el artefacto se encapsula en una imagen de producción **ligera** (multietapa: solo JRE + `.jar` en runtime), **segura** (non-root) y con **cuotas estrictas de hardware** (cgroups calibrados).

---

2. Tabla de Identidad

Identidad	Comando	Salida
Hash de la imagen	`docker inspect --format='{{.Id}}' eco-air-backend:1.0.0`	`sha256:5b7313eb3588d77c764055432e60b927c7c223d07ebb708839a2d58405275a06`

Identidad	Comando	Salida
Variable de entorno inyectada	`docker exec eco_air_active env | grep OPERATOR_ID`	`OPERATOR_ID=871752`
Consumo final calibrado (umbral 256m)	`docker stats --no-stream eco_air_active`	ver tabla a continuación

Consumo final calibrado (`docker stats --no-stream eco_air_active`, límite 256m)

```
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
421eb161867b   eco_air_active   0.39%     174.5MiB / 256MiB   68.17%    1.17kB / 126B   0B / 0B     48
```

- **CPU %**: 0.39 %
- **MEM USAGE / LIMIT**: 174.5 MiB / 256 MiB
- **MEM %**: 68.17 %
- **PIDS**: 48

---

3. Reto 1 — Construcción Multietapa con Huella Estricta

Comando de construcción obligatorio

```
docker build --build-arg STUDENT_ID="871752" -t eco-air-backend:1.0.0 .
```

Capa por capa del Dockerfile

Etapa	Instrucción	Propósito (seguridad / optimización)
builder	`FROM maven:3.9.6-eclipse-temurin-17-alpine AS builder`	Compila dentro del contenedor; no requiere Maven local. Imagen base Alpine = menor huella.
builder	`COPY pom.xml .` + `RUN mvn dependency:go-offline -B`	Aislar la resolución de dependencias **antes** de copiar `src/` preserva la caché: los rebuilds de código no vuelven a descargar el árbol de dependencias.
builder	`ARG STUDENT_ID`	Argumento de compilación inyectado con `--build-arg`.
builder	`RUN mvn clean package -DskipTests -B`	Empaqueta el `.jar` omitiendo pruebas (las pruebas van en CI, no en la imagen de producción).
runtime	`FROM eclipse-temurin:17-jre-alpine`	Solo JRE: ni JDK, ni Maven, ni fuentes. Menor superficie de ataque.
runtime	`RUN addgroup -g 10001 sensorgroup && adduser -u 10001 ... sensor_user`	Usuario sin privilegios con UID/GID fijos (predecibles para la auditoría `docker top`).

Etapa	Instrucción	Propósito (seguridad / optimización)
runtime	`COPY --from=builder /app/target/eco-air-backend.jar app.jar`	Copia **únicamente** el `.jar`. No se traen fuentes ni cache de Maven.
runtime	`ENV JAVA_OPTS=...`	Flags ergonómicos: `MaxRAMPercentage=75` respeta el cgroup; `ExitOnOutOfMemoryError` deja morir a la JVM limpio; `UseG1GC`; entropía rápida.
runtime	`ENV OPERATOR_ID=${STUDENT_ID}`	El ID del estudiante queda disponible en runtime (Tabla de Identidad).
runtime	`RUN chown -R sensor_user:sensorgroup /app`	Titularidad de `/app` para el usuario sin privilegios.
runtime	`USER sensor_user`	Cambia a non-root **antes** del ENTRYPOINT: el `java` arranca como `sensor_user`.
runtime	`ENTRYPOINT ["sh","-c","java $JAVA_OPTS -jar app.jar"]`	Arranque con expansión de `JAVA_OPTS`.

> Para que `OPERATOR_ID` recibiera el valor del `--build-arg` en la etapa runtime, el `ARG STUDENT_ID` se declaró **antes del primer** `FROM` (global) y se **re-declaró** en la etapa runtime. Los `ARG` declarados solo en una etapa no son visibles en otra.

Verificación del desacoplamiento de capas

```
docker history --human --format "table {{.Size}}\t{{.CreatedBy}}" eco-air-backend:1.0.0
```

```
SIZE      CREATED BY
0B        ENTRYPOINT ["sh" "-c" "java $JAVA_OPTS -jar app.jar"]
0B        USER sensor_user
48.4MB    RUN |1 STUDENT_ID=871752 /bin/sh -c chown -R sensor_user:sensorgroup /app
0B        ENV OPERATOR_ID=871752
0B        ARG STUDENT_ID=871752
0B        ENV JAVA_OPTS=-XX:InitialRAMPercentage=50 -XX:MaxRAMPercentage=75 ...
48.4MB    COPY /app/target/eco-air-backend.jar app.jar
4.1kB     WORKDIR /app
36.9kB    RUN /bin/sh -c addgroup -g 10001 sensorgroup && adduser -u 10001 ...
0B        ENTRYPOINT ["/__cacert_entrypoint.sh"]
12.3kB    COPY --chmod=755 entrypoint.sh /__cacert_entrypoint.sh
12.3kB    RUN /bin/sh -c set -eux; echo "Verifying install ..." ...
141MB     RUN /bin/sh -c set -eux; ARCH="$(apk --print-arch)" ... (instalacion del JRE)
0B        ENV JAVA_VERSION=jdk-17.0.20.1+1
44MB      RUN /bin/sh -c set -eux; apk add --no-cache fontconfig ...
0B        ENV LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8
0B        ENV PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
0B        ENV JAVA_HOME=/opt/java/openjdk
0B        CMD ["/bin/sh"]
9.08MB    ADD alpine-minirootfs-3.24.2-x86_64.tar.gz /
```

Lectura: las capas del builder (Maven, fuentes, cache de dependencias) **no aparecen** en la imagen final. Solo se hereda la capa `COPY ... app.jar` (48.4 MB) desde el builder; el resto pertenece a la imagen base `eclipse-temurin:17-jre-alpine`. El desacoplamiento es efectivo.

---

4. Reto 2 — Auditoría Forense y Comprobación de Vulnerabilidades

4.1 Verificación del Desacoplamiento de Capas

Comando y salida: ver §3 (bloque `docker history` de arriba). Se confirma que la imagen final no contiene capas del builder salvo el `.jar`.

4.2 Inspección del Proceso del Host (Aislamiento de Privilegios)

```
docker run -d --name eco_air_active -p 8080:8080 eco-air-backend:1.0.0
docker top eco_air_active -o pid,user,comm
```

```
PID                 USER                COMMAND
2807                10001               java
```

Evidencia: el comando `java` **no** se ejecuta como `root`. Se ejecuta bajo el identificador `10001`, que corresponde a `sensor_user` (UID 10001, creado en el Dockerfile). El aislamiento de privilegios funciona: aunque el contenedor se arranque sin `--user`, el `USER sensor_user` del Dockerfile hace que el PID 1 sea non-root.

Arranque confirmado en los logs:

```
2026-10-20T21:26:55.575Z  INFO 1 --- [eco-air-backend] [           main]
  com.ecoair.EcoAirApplication : Started EcoAirApplication in 4.683 seconds
  (process running for 5.224)
```

4.3 Test de Penetración: Inmutabilidad de Runtime

```
docker exec eco_air_active touch /etc/crontab_fake
```

```
touch: cannot touch '/etc/crontab_fake': Permission denied
```

Evidencia: el intento de escribir un script malicioso en `/etc` falla con `Permission denied`. Como el proceso corre como `sensor_user` (sin privilegios), no puede modificar directorios del sistema (`/etc`). La inmutabilidad de runtime queda demostrada: un atacante que comprometa la JVM no podría, por ejemplo, instalar una tarea en `cron`.

4.4 Auditoría de Artefactos de Desarrollo

```
docker exec eco_air_active which mvn
docker exec eco_air_active find / -name "*.java" 2>/dev/null
```

```
(salida vacía)
(salida vacía)
```

Evidencia y explicación: - `which mvn` no devuelve nada: **Maven no está** en la imagen runtime. Maven solo existe en la etapa builder (imagen `maven:...`), que se descarta. La imagen final usa `eclipse-temurin:17-jre-alpine`, que solo contiene el JRE. - `find / -name "*.java"` no devuelve nada: **no hay código fuente** `.java` en la imagen runtime. Solo se copió el `.jar` compilado.

Conclusión: la superficie de ataque se reduce al mínimo. No hay herramientas de build ni código fuente que un atacante pudiera inspeccionar o recompilar dentro del contenedor.

---

5. Reto 3 — Calibración de cgroups y Captura del Asesino OOM

5.1 Inducción del Colapso por Memoria (OOMKilled)

```
docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0
# (esperar a que el cgroup colapse)
docker inspect eco_air_crash --format='Status: {{.State.Status}} | ExitCode: {{.State.ExitCode}} | OOMKilled
```

```
Status: exited | ExitCode: 137 | OOMKilled: true
```

Logs del contenedor crash (no llega a arrancar):

```
2026-10-20T21:27:03.332Z  INFO 1 --- [eco-air-backend] [           main]
  com.ecoair.EcoAirApplication : Starting EcoAirApplication v1.0.0 ...
2026-10-20T21:27:04.067Z  INFO 1 --- [eco-air-backend] [           main]
  com.ecoair.EcoAirApplication : No active profile set, falling back to 1 default profile: "default"
```

Evidencia y explicación: con un límite de **48 MB**, el contenedor no logra levantar Spring Boot + Hibernate (el log se detiene en "No active profile set"). La JVM agota la cuota del cgroup y el **OOM killer del kernel** termina el proceso con **SIGKILL**, lo que se refleja en `ExitCode: 137` (128 + 9) y `OOMKilled: true`. Esto valida el comportamiento esperado: el sistema operativo mata el proceso que excede su cuota en lugar de comprometer al resto del servidor de la estación meteorológica.

5.2 Calibración del Límite Operativo Mínimo

Se iteró el parámetro `-m` (incluyendo valores intermedios para hallar el umbral **exacto**). Para cada límite se verificó: (1) inicio correcto, (2) arranque en menos de 15 s según `docker logs --timestamps ... | grep "Started EcoAirApplication"`, y (3) profiling de memoria en caliente estable con `docker stats --no-stream`.

```
docker logs --timestamps eco_air_active | grep "Started EcoAirApplication"
docker stats --no-stream eco_air_active
```

`-m`	¿Inicia?	Tiempo arranque	`Started EcoAirApplication`	MEM USAGE / LIMIT	MEM %	CPU %	Veredicto
48m	No (OOM)	—	no aparece	—	—	—	Descartado: OOMKilled (exit 137)
128m	Sí (al límite)	~14 s	13.451 s	120.8 MiB / 128 MiB	94.40 %	0.36 %	Descartado: al borde de 15 s y 94 % MEM con swap/bloque activo
192m	Sí	~4 s	3.614 s	169.6 MiB / 192 MiB	88.33 %	0.34 %	Arranca, pero 88 % MEM deja margen estrecho ante ráfagas
256m	Sí	~5 s	4.053 s	216.5 MiB / 256 MiB	84.56 %	0.43 %	**Umbral mínimo recomendado**

`-m`	¿Inicia?	Tiempo arranque	`Started EcoAirApplication`	MEM USAGE / LIMIT	MEM %	CPU %	Veredicto
384m	Sí	~4 s	3.622 s	189.8 MiB / 384 MiB	49.42 %	0.35 %	Estable, margen holgado
512m	Sí	~4 s	3.856 s	210.5 MiB / 512 MiB	41.10 %	0.27 %	Estable, margen holgado

Umbral mínimo exacto hallado: `256m`.

Justificación: - `48m` y por debajo: el contenedor es matado por el OOM killer (no inicia). - `128m`: la app inicia, pero tarda **13.451 s**, rozando el límite de 15 s, y consume el 94 % de la cuota con actividad de swap/bloque elevada (`132 MB / 172 MB`). Su tiempo de arranque oscila entre corridas (13-17 s), por lo que no cumple el criterio de arranque confiable < 15 s ni de estabilidad. - `192m`: arranca en 3.6 s (< 15 s), pero con 88 % de uso de memoria el margen es demasiado estrecho para las ráfagas continuas de métricas del caso de uso. - `256m`: arranca en 4.05 s (< 15 s) y, en caliente, se estabiliza en torno al 68 % de MEM (ver stats final), con margen sano y estable ante tráfico.

Tabla resultante de `docker stats` (umbral calibrado 256m)

```
docker stats --no-stream eco_air_active
```

```
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
421eb161867b   eco_air_active   0.39%     174.5MiB / 256MiB   68.17%    1.17kB / 126B   0B / 0B     48
```

- **CPU %**: 0.39 %
- **MEM USAGE / LIMIT**: 174.5 MiB / 256 MiB
- **MEM %**: 68.17 %

---

6. Conclusiones

1. **Huella estricta (Reto 1).** La construcción multietapa produce una imagen runtime que contiene únicamente el JRE y el `.jar`. La auditoría `docker history` confirma que las capas del builder (Maven, fuentes, cache) no forman parte de la imagen final.
2. **Aislamiento de privilegios (Reto 2).** El proceso `java` corre como `sensor_user` (UID 10001), no como `root`. La prueba de escritura en `/etc` falla con `Permission denied`, y la imagen runtime no contiene Maven ni archivos `.java`.
3. **Estrés de cgroups (Reto 3).** Con 48 MB el OOM killer mata el contenedor (`ExitCode: 137`, `OOMKilled: true`). La calibración iterativa estableció **256 MB** como el umbral operativo mínimo en el que la app arranca en menos de 15 s y se mantiene estable con margen suficiente para las ráfagas de métricas de los sensores.

El encapsulamiento cumple los tres objetivos del caso: imagen **ligera**, **segura** (non-root) y con **cuotas estrictas de hardware** (cgroups calibrados), lista para la orquestación multicapa.